VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Report

Week 5

Course: Applications of Computer Graphics

Supervisors:
Nguyễn Trọng Việt
Phạm Thanh Tùng
Võ Thế Hào

Student:
23127266 - Nguyễn Anh Thư

March 20, 2026

Acknowledgment

I would like to express sincere gratitude to Nguyen Trong Viet, Pham Thanh Tung and Vo The Hao for their guidance and support throughout the course.

Contents

Chapter I: Introduction **1**

Chapter II: Access, Setup, and Usage **2**

1 Access 2

2 Local setup 2

3 Usage 2

Chapter III: Implementation **3**

1 Interface structure 3

2 Client-side functions used 3

3 Extracting frames (pixel buffers) 3

4 Pixel-level edge detection (Canny-style) 3

5 Real-time scheduling 3

Chapter IV: Processing Flow **4**

Chapter V: Limitations and Future Work **5**

Chapter VI: Conclusion **6**

Reference **7**

Chapter I: Introduction

This project builds a simple web application using HTML5 `<video>` and Canvas to perform real-time edge detection on every frame while the video is playing. All processing runs locally in the browser with client-side JavaScript, and the detected edges are rendered immediately beside the original video.

Chapter II: Access, Setup, and Usage

1. Access

- **GitHub repository:** https://github.com/ariesanhthu/homework5
- **GitHub Pages:** https://ariesanhthu.github.io/homework5/

2. Local setup

The app can be tested locally by installing dependencies and starting the Node.js server (then opening the local address in a browser).

3. Usage

Select a local video file, press `Play`, and observe two synchronized views: the original video on one side and the edge map on the Canvas output on the other side. Processing automatically pauses when the video is paused or ended.

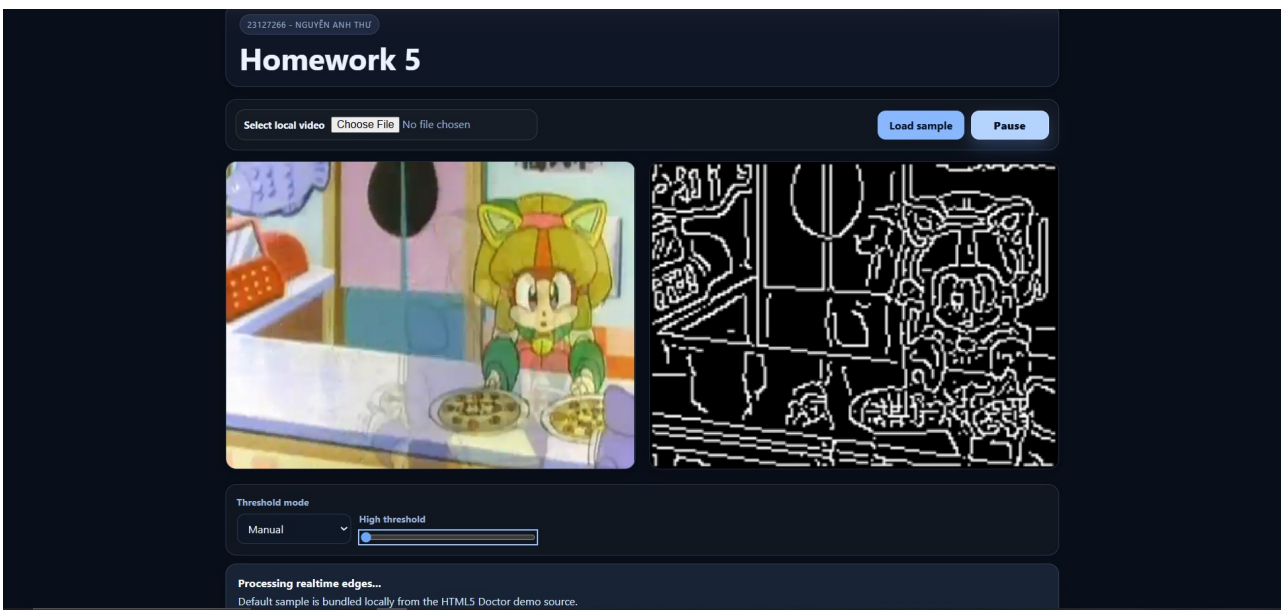

Figure II.1: Example edge output from the real-time pipeline (per-frame processing).

Chapter III: Implementation

1. Interface structure

The UI is centered around a video element as the input source and a Canvas element as the output surface. An offscreen Canvas is used internally for pixel processing, while the visible Canvas displays the final edge image.

2. Client-side functions used

The main workflow is driven by video playback events and a real-time render loop. Key browser/Canvas functions include: `play()`, `pause()`, `addEventListener()`, `URL.createObjectURL()`, `getContext('2d')`, `drawImage()`, `getImageData()`, `putImageData()`, `clearRect()`, `requestAnimationFrame()`, and `cancelAnimationFrame()`.

3. Extracting frames (pixel buffers)

Instead of processing the raw video file directly, the system processes the *currently displayed frame*. In each loop iteration, the current video frame is drawn onto the internal processing Canvas via `drawImage()`, then pixel data is retrieved with `getImageData()`. After the edge pipeline finishes, the processed edge buffer is written back with `putImageData()` and the visible Canvas is updated for the user.

4. Pixel-level edge detection (Canny-style)

The edge detection pipeline is a compact Canny-style implementation optimized for per-frame processing:

1. **Grayscale conversion:** RGB is converted to intensity using integer weights equivalent to $0.299R + 0.587G + 0.114B$ (implemented with weights `77/150/29` and a shift for speed).

2. **Noise reduction:** a separable Gaussian blur is applied with a 5-tap kernel shaped as $[1, 4, 6, 4, 1]/16$ using two passes (horizontal then vertical) to reduce computation.

3. **Gradient estimation (Sobel):** gradients in x and y directions are computed using the Sobel stencil, then the gradient magnitude is measured (using `hypot`) and the gradient direction is quantized into 4 main bins: 0°, 45°, 90°, and 135°.

4. **Non-maximum suppression (NMS):** for each interior pixel, the magnitude is compared with two neighbors aligned to the quantized direction; only local maxima are kept, suppressing thick edges.

5. **Thresholding + hysteresis:** thresholds are resolved either in *manual* mode or *adaptive* mode. Pixels classified as *strong* ($\geq$ high threshold) start a stack-based propagation; *weak* pixels (between low and high) are promoted to strong if connected in the 8-neighborhood. The output is a binary edge map encoded as intensity values (0 or 255).

5. Real-time scheduling

The processing loop uses `requestAnimationFrame()` to stay synchronized with browser rendering. To reduce redundant work, frames are skipped when the video timestamp changes less than a small interval (approximately targeting 120 FPS processing behavior).

Chapter IV: Processing Flow

Video Input → Extract Current Frame → Grayscale → Separable Gaussian Blur → Sobel Gradients → Quantize Dire

Chapter V: Limitations and Future Work

- Performance depends on processing resolution and device capabilities (larger videos may reduce frame rate).
- Edge quality can be sensitive to threshold configuration and to the selected blur scale.
- The implementation is designed for interactivity and may not match the accuracy of specialized optimized vision libraries.

Possible improvements include using *OffscreenCanvas* or *Web Workers* to move pixel processing off the main thread, and adding more parameter controls (e.g., dynamic thresholding strategies tuned to scene statistics).

Chapter VI: Conclusion

Overall, this project demonstrates a real-time, local edge detection website using HTML5 Video and Canvas. The pipeline performs grayscale conversion, separable Gaussian blur, Sobel gradients, non-maximum suppression, and hysteresis thresholding directly on pixel buffers extracted from the video element.

Reference

[1] Application graphics: Web graphics environment. Week 7–8 lecture slides, 2025.